02

MGU BCA - 3RD SEM

Feature Engineering

Module 2 - Numerical, Categorical, Selection

By Aben Sojan

Exam - 18 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

FE theory exam: 35 marks, 1 hour (MG3DSEBCA201 Honours model paper, mgu.ac.in Oct 2025).

Printed: 10 x Part A (2 marks) + 5 x Part B (5 marks) = 45 marks printed, so there IS choice.

Likely: Part A all compulsory (20) + answer any 3 of 5 in Part B (15) = 35. Exact choice rule UNVERIFIED - confirm from the paper.

Module 2 gave 4 of 10 Part A + 3 of 5 Part B (5-mark: RFE, extraction vs selection, interaction features) in the model paper.

Strongest cross-paper repeats: feature selection methods (3x), dimensionality reduction (3x), PCA steps (2x), extraction vs selection (2x).

TOPIC 1

Binning and Discretization

1

Binning groups continuous numbers into fixed intervals called bins.

Example: ages 0-100 become groups like 0-18, 19-35, 36-60, 61+.

It reduces noise and helps models find patterns inside ranges.

Two ways: equal-width bins (same interval size) and equal-frequency bins (same count per bin).

QUESTION BANK

1 What is binning? Explain with an example.

IMPORTANT

Definition: Binning is the process of grouping continuous numeric values into a small number of fixed intervals called bins.

Each bin gets a label, turning numbers into categories the model can handle easily.

Example: ages from 0 to 100 are grouped as 0-18 Child, 19-35 Youth, 36-60 Adult and 61+ Senior.

A customer aged 25 falls in the Youth bin, and the model uses the bin label instead of the exact age.

Binning reduces the effect of small noise, since ages 24 and 26 are treated the same.

It also helps models find patterns inside ranges, like young customers buy more.

The two main methods are equal-width bins with the same interval size and equal-frequency bins with the same record count.

In short, binning trades fine detail for stable, meaningful groups.

2 Distinguish between equal-width and equal-frequency binning.

IMPORTANT

Both are binning methods, but they divide the data using different rules.

Equal-width binning splits the value range into bins of the same size, like 0-25, 25-50, 50-75 and 75-100.

It is simple and keeps the ranges easy to explain, but skewed data can leave some bins nearly empty.

Equal-frequency binning puts the same number of records in each bin, so bin ranges stretch where data is sparse.

It handles skewed data better because every bin gets a fair share of records.

Example: for income data, equal-width bins may pile everyone into the first bin, while equal-frequency bins spread them evenly.

In pandas, cut() makes equal-width bins and qcut() makes equal-frequency bins.

In short, use equal-width for simple even data and equal-frequency for skewed data.

3 What is discretization? How is it related to binning?

IMPORTANT

Definition: Discretization is the general process of converting continuous data into discrete groups or categories.

Continuous values like 72.4 marks become labels like First Class, which models handle more easily.

Binning is the most common discretization method: it cuts the range into intervals called bins.

So binning is one way of doing discretization, just as cut() is one way of binning in pandas.

Other discretization ideas include grouping by business rules, like tax slabs defined by the government.

The goal is always the same: reduce noise and make patterns clearer.

Discretization is widely used before models that prefer categories, like decision trees and naive Bayes.

In short, discretization is the concept and binning is its most popular technique.

TOPIC 2

2

Polynomial and Interaction Features

Polynomial features add powers of a feature, like x squared, to catch curves.

Interaction features multiply two features, like area = length x breadth.

They help simple linear models learn non-linear relationships.

Too many polynomial features can cause overfitting.

QUESTION BANK

4 What are polynomial features? Why are they used?

IMPORTANT

Definition: Polynomial features are new columns made from powers of existing features, like x^2 and x^3.

They are used because simple linear models can only draw straight lines, but real data often curves.

Adding x^2 lets a linear model fit a curve while staying a linear model in the new features.

Example: house price may grow faster than size, so adding size-squared captures that curve.

In sklearn, PolynomialFeatures(degree=2) creates these columns automatically.

They are also useful when the true relationship is known to be curved, like projectile motion.

The danger is overfitting: too many powers make the model memorise noise instead of the trend.

In short, polynomial features give simple models the power to bend.

5 What is an interaction feature? Give an example.

IMPORTANT

Definition: An interaction feature combines two or more features, usually by multiplying them, to capture their joint effect.

Some effects appear only when features act together, and single features miss them completely.

Example: rooms x area captures house size better than rooms and area separately.

A 3-room flat of 500 sq ft and a 3-room flat of 1500 sq ft are very different, and only the product shows it.

Another example: in marketing, age group x income level can reveal that young high-earners behave uniquely.

Interaction features are created manually by multiplication or automatically by PolynomialFeatures with interaction_only=True.

Like polynomial features, they must be used carefully to avoid overfitting.

In short, interaction features let the model see teamwork between columns.

6 What precautions should be taken while creating polynomial features?

IMPORTANT

Polynomial features are powerful, but careless use quickly damages the model.

Precaution 1: create them only when the relationship actually looks curved; adding powers to straight-line data adds pure noise.

Precaution 2: keep the degree low, usually 2 or 3, because higher degrees overfit very fast.

Precaution 3: always scale features first, since x^2 of an unscaled large number explodes in size.

Precaution 4: watch the feature count; degree 3 on 10 features creates hundreds of columns.

Precaution 5: use regularization like Ridge or Lasso to control the extra flexibility.

Precaution 6: validate on test data to confirm the curves reflect reality, not noise.

In short, use low degrees, scale first, and always check against overfitting.

TOPIC 3

One-Hot Encoding

3

One-hot encoding turns each category into its own 0/1 column.

Example: colour Red, Green, Blue becomes three columns with one 1 and rest 0.

Best for nominal data with few categories and no order.

Problem: too many categories create too many columns.

7 What is one-hot encoding? Explain with an example.

IMPORTANT

Definition: One-hot encoding converts each category of a column into its own new column containing only 0 or 1.

For every row, the column of its own category gets 1 and all other new columns get 0.

Example: a Colour column with Red, Green, Blue becomes three columns: Colour_Red, Colour_Green, Colour_Blue.

A red shirt becomes 1, 0, 0; a green shirt becomes 0, 1, 0; a blue shirt becomes 0, 0, 1.

This is why it is called one-hot: exactly one column is hot (1) per row.

It is best for nominal data with few categories and no natural order, like gender or city.

It avoids the false ordering problem of label encoding, since no category looks greater than another.

The price is extra columns, so it suits columns with a small number of categories.

8 What is the main disadvantage of one-hot encoding?

IMPORTANT

The main disadvantage is the explosion of columns when a category has many unique values.

A column with 1000 cities becomes 1000 new columns, most of them zeros.

This wastes memory and slows down training considerably.

It also makes the data very sparse, which weakens many models.

This problem is known as the curse of dimensionality.

One common fix is to drop one column to avoid the dummy variable trap in linear models.

Better fixes for high-cardinality data are hashing or target encoding.

In short, one-hot encoding is excellent for few categories but breaks down for many.

9 Write code for one-hot encoding in pandas and sklearn.

IMPORTANT

One-hot encoding is a one-line operation in both pandas and sklearn.

In pandas, get_dummies() converts the chosen columns instantly:

```
df = pd.get_dummies(df, columns=["city"])
```

This replaces the city column with one 0/1 column per city.

In sklearn, OneHotEncoder fits into ML pipelines:

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False)
X_new = enc.fit_transform(df[["city"]])
```

The sklearn version remembers the categories, so it encodes new data consistently during prediction.

Use pandas for quick analysis and sklearn when building a proper training pipeline.

TOPIC 4

Label Encoding

4

Label encoding gives each category a number: Red=0, Green=1, Blue=2.

It is simple and keeps only one column.

Best for ordinal data where order matters, like Poor=0, Average=1, Good=2.

Danger: the model may wrongly think Blue(2) is greater than Red(0).

QUESTION BANK

10 What is label encoding? Write the sklearn code.

IMPORTANT

Definition: Label encoding replaces each category with an integer label, keeping everything in a single column.

Example: sizes S, M, L become 0, 1, 2.

It is simple, compact and needs no extra columns.

The sklearn code is:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["size"] = le.fit_transform(df["size"])
```

fit_transform learns the mapping on training data; transform() alone is used on new data.

The original labels can be recovered with le.inverse_transform() when needed.

In short, label encoding is the compact way to number categories.

11 When should label encoding be used, and when avoided?

IMPORTANT

Use label encoding for ordinal data, where the order of categories is real and meaningful.

Example: grades Poor=0, Average=1, Good=2, where the numbers correctly reflect increasing quality.

Tree-based models also tolerate label-encoded nominal data fairly well, since they split on thresholds.

Avoid it for nominal data with linear models, because the numbers create a false order.

Example: encoding cities as 1, 2, 3 tricks a linear model into treating city 3 as greater than city 1.

In such cases one-hot encoding is the safer choice.

The decision always depends on both the data type and the model being used.

In short, use label encoding when order is real, avoid it when order would be invented.

TOPIC 5

One-Hot vs Label Encoding

5

One-hot creates many 0/1 columns; label encoding creates one numbered column.

Use one-hot for nominal data with few categories; label encoding for ordinal data.

One-hot avoids false ordering; label encoding is compact and tree-friendly.

For high-cardinality data like 500 cities, both struggle; hashing or target encoding is used.

QUESTION BANK

12 Compare one-hot encoding and label encoding.

IMPORTANT

Both convert categories to numbers, but they do it in opposite ways.

One-hot encoding creates many columns with 0/1 values: one new column per category.

Label encoding creates a single column with integer values, one number per category.

One-hot avoids false ordering completely, since no category looks larger than another.

Label encoding is compact and memory-friendly, and it preserves real order in ordinal data.

Example: Gender M/F suits one-hot encoding; Grade A/B/C suits label encoding.

One-hot struggles with many categories due to column explosion; label encoding struggles with nominal data due to false order.

In short, one-hot is safe but wide, label encoding is compact but order-sensitive.

13 A 'city' column has 500 cities. Which encoding would you choose and why?

IMPORTANT

A city column with 500 unique values is called high-cardinality data, and it needs special care.

One-hot encoding would create 500 new columns, which wastes memory and slows training badly.

Label encoding keeps just one column, which is far more practical here.

Tree-based models like random forests handle label-encoded cities reasonably well.

For linear models, even better options are hashing encoding or target encoding, which compress the cities smartly.

The choice also depends on the model: trees tolerate label encoding, linear models prefer smarter encodings.

Blindly one-hot encoding 500 cities is one of the most common beginner mistakes.

In short, avoid one-hot for 500 cities; use label encoding for trees or target/hashing encoding for linear models.

TOPIC 6

Feature Extraction vs Feature Selection

6

Feature selection picks the best existing features and drops the rest.

Feature extraction creates new smaller features from old ones, like PCA.

Selection keeps the original meaning; extraction loses it but compresses better.

Both aim to reduce dimensions, speed up training and improve the model.

QUESTION BANK

14 **Distinguish between feature extraction and feature selection. (FE model paper; MCA ML Oct 2025)**

REPEATED

Both reduce the number of features, but they do it in fundamentally different ways.

Feature selection chooses a subset of the original features and drops the rest.

Example: keeping only age and income from 20 columns and deleting the other 18.

Feature extraction builds brand-new features by combining the old ones.

Example: PCA merges 20 columns into 3 new components that capture most of the information.

Selection keeps the original column names and meanings, so results stay easy to explain.

Extraction usually compresses better when features are highly correlated, but the new features have no simple meaning.

In short, selection picks the best old features, extraction invents new compact ones.

15 **When would you prefer feature selection over extraction?**

IMPORTANT

Prefer feature selection when interpretability matters, because the original column names are kept.

Example: a bank must explain why a loan was rejected, so it needs real features like income and credit score, not PCA components.

Selection is also simpler to implement and easier to debug.

It works well when a few features are clearly useful and the rest are noise.

Prefer extraction when maximum compression is the goal and meaning does not matter.

Example: compressing 1000 image pixels into 50 components for faster training.

Extraction also helps when features are highly correlated and selection cannot separate them cleanly.

In short, choose selection for explainable results and extraction for maximum compression.

TOPIC 7

7

Steps in Feature Selection

Step 1: generate candidate feature subsets to try.

Step 2: evaluate each subset with a scoring method or model accuracy.

Step 3: stop when the score stops improving or a limit is reached.

Step 4: validate the chosen subset on fresh test data.

QUESTION BANK

16 **Explain the steps involved in feature selection.**

IMPORTANT

Feature selection follows a systematic search process in four stages.

Stage 1, subset generation: create candidate feature subsets to try, like {age}, {age, income} and {age, income, city}.

Stage 2, subset evaluation: score each subset using a statistical measure or the actual model accuracy.

Stage 3, stopping criterion: halt the search when scores stop improving or a time limit is reached.

Stage 4, validation: confirm the chosen subset performs well on fresh unseen test data.

This process can be forward (adding features), backward (removing features) or exhaustive for small data.

Skipping validation is dangerous, because a subset can look good on training data but fail on new data.

In short, generate, evaluate, stop sensibly and validate.

17 **What are the four stages of the feature selection process?**

IMPORTANT

The four standard stages of feature selection are generation, evaluation, stopping and validation.

Generation: create candidate subsets of features to examine.

Evaluation: score each subset with a metric, a statistical test or model performance.

Stopping: halt the search when the score stops improving, a target size is reached, or time runs out.

Validation: test the final selected subset on unseen data to confirm it truly works.

These stages apply whether the method is filter, wrapper or hybrid.

Missing the stopping stage wastes time; missing validation risks overfitting.

In short, the four stages are generate, evaluate, stop and validate.

18 Why is a stopping criterion needed in feature selection?

IMPORTANT

A stopping criterion tells the search when to halt, and without it the process has no natural end.

With n features there are 2^n possible subsets, so an exhaustive search quickly becomes impossible.

Example: 30 features give over a billion subsets, which no computer can try fully.

A good stopping rule saves huge amounts of time by quitting once scores stop improving.

Example: stop when accuracy does not improve for 3 rounds in a row.

It also prevents overfitting, because endlessly chasing tiny training gains usually hurts new data.

Common rules are a fixed number of features, a score threshold or a time budget.

In short, the stopping criterion keeps the search fast, practical and safe.

TOPIC 8

Filter, Wrapper and Hybrid Methods

8

Filter methods score features with statistics like correlation or chi-square, without any model; they are fast.

Wrapper methods try feature subsets with the actual model, like forward selection and RFE; accurate but slow.

Hybrid (embedded) methods select features during training, like Lasso and tree feature importance.

Use filter for speed on large data; wrapper for best accuracy on small data.

QUESTION BANK

19 **Explain filter, wrapper and hybrid feature selection methods. (IMCA DS Dec 2025, asked twice; MCA ML Oct 2025)**

SURE MARK

These are the three families of feature selection, differing in how they judge features.

Filter methods rank features using statistics alone, with no model involved.

Example: keep features with high correlation to the target, or use the chi-square test for categories.

They are very fast and work well on large data, but they may miss interactions between features.

Wrapper methods train the actual model on many feature subsets and keep the best-scoring subset.

Example: recursive feature elimination (RFE) repeatedly drops the weakest feature.

They are accurate because they test with the real model, but they are slow on large data.

Hybrid or embedded methods perform selection inside training itself; example: Lasso regression shrinks useless feature weights to zero, and random forests rank features by importance.

20 **Compare filter and wrapper methods with examples.**

IMPORTANT

Filter and wrapper methods differ in speed, accuracy and how they use the model.

Filter methods use only statistics, so they are fast and model-independent.

Example: a chi-square filter quickly ranks 1000 gene columns without training anything.

But filters judge features one by one and can miss useful combinations.

Wrapper methods train the real model on each candidate subset, so they are highly accurate.

Example: RFE with logistic regression on 20 columns finds the subset that truly performs best.

The price is speed: wrappers are far too slow for thousands of features.

In short, use filters for speed on big data and wrappers for accuracy on small data.

21 What is forward selection and backward elimination?

IMPORTANT

Both are wrapper-style search strategies that build the feature set step by step.

Forward selection starts with an empty set and adds the best feature one at a time.

After each addition the model is scored, and the process stops when the score stops improving.

Backward elimination works in reverse: it starts with all features and removes the weakest one at a time.

It also stops when removing more features starts hurting the score.

Forward selection is cheaper when the final set is small; backward elimination sees feature interactions better at the start.

Both are greedy methods, meaning they never reconsider an earlier choice.

In short, forward selection builds up and backward elimination trims down.

22 What are embedded methods? Give examples.

IMPORTANT

Definition: Embedded methods perform feature selection as a natural part of model training.

Unlike filters and wrappers, there is no separate selection step; the model selects while it learns.

Example 1: Lasso regression adds a penalty that forces the weights of useless features to exactly zero.

The surviving non-zero weights reveal the selected features automatically.

Example 2: random forests rank features by importance during training, so the lowest-ranked ones can be dropped.

Embedded methods are faster than wrappers because selection and training happen together.

They are also more accurate than filters because the real model guides the choice.

In short, embedded methods select features as a side effect of training.

23 **Explain how Recursive Feature Elimination (RFE) works as a wrapper method for feature selection.**

IMPORTANT

Definition: RFE is a wrapper method that finds the best features by repeatedly training a model and dropping the weakest feature.

It is called a wrapper because it wraps around an actual machine learning model to judge features.

Step 1: train the model on all features and rank them by importance, like the weights in logistic regression.

Step 2: remove the single weakest feature from the set.

Step 3: retrain the model on the remaining features and rank again.

These steps repeat until only the desired number of features is left.

Example: from 20 columns, RFE with logistic regression can narrow down to the 5 columns that truly drive predictions.

It is accurate because the real model judges every subset, but slow on large data since it retrains many times.

In short, RFE eliminates features one by one until only the strongest survive.

TOPIC 9

9

PCA - Introduction and Application

PCA (Principal Component Analysis) reduces dimensions by finding new axes called principal components.

The first component captures the maximum variance in the data.

Used for visualization, noise reduction and faster training.

Steps: standardize data, compute covariance matrix, find eigenvectors, pick top components, project data.

QUESTION BANK

24 **What is dimensionality reduction? Why is it important? (IMCA DS Dec 2025; MCA ML Dec 2023, asked twice)**

SURE MARK

Definition: Dimensionality reduction is the process of reducing the number of features in a dataset while keeping as much information as possible.

High-dimensional data, like images with thousands of pixels, is slow to process and hard to visualise.

It is important first for speed: fewer features mean faster training and less memory.

Second, it fights overfitting: extra useless dimensions give the model more noise to memorise.

Third, it enables visualisation: reducing to 2 or 3 dimensions lets us actually plot and see the data.

Two main routes exist: feature selection, which picks the best original features, and feature extraction like PCA, which builds new compact ones.

Example: 100 correlated sensor readings compressed to 5 PCA components keep nearly all the useful signal.

In short, dimensionality reduction keeps the signal, drops the bulk, and makes models faster and clearer.

25 **What is PCA? Explain its steps. (IMCA DS Dec 2025; MCA ML Oct 2025)**

REPEATED

Definition: PCA (Principal Component Analysis) is a feature extraction technique that reduces dimensions by finding new uncorrelated axes called principal components.

The first component captures the maximum variance (spread) in the data, the second captures the next most, and so on.

It is used for visualization, noise reduction and faster model training.

Step 1: standardize all features so that large-range features do not dominate.

Step 2: compute the covariance matrix, which shows how features vary together.

Step 3: find its eigenvectors and eigenvalues; the eigenvectors are the new axes and eigenvalues show their importance.

Step 4: keep the top components that capture most of the variance and project the data onto them.

Example: 100 correlated sensor readings can be compressed into 5 components with little information loss.

26 Why is standardization needed before PCA?

IMPORTANT

PCA looks for the directions of maximum variance, so scale differences would hijack the result.

A feature with a huge range, like salary in lakhs, has far more variance than age in tens, purely because of units.

Without standardization, the first principal component would simply point along salary and ignore everything else.

Standardization puts every feature on the same scale, with mean 0 and standard deviation 1.

After that, variance truly reflects information content, not measurement units.

Example: without scaling, salary crushes age in the components; with scaling, both contribute fairly.

This is why every PCA pipeline starts with StandardScaler.

In short, standardization stops big units from bullying small units in PCA.

27 What are principal components?

IMPORTANT

Definition: Principal components are the new axes created by PCA from combinations of the original features.

Each component is a weighted mix of all original features, pointing in a direction of high data spread.

They are uncorrelated with each other, meaning each one captures fresh information the others missed.

The first component holds the maximum possible variance, the second holds the next most, and so on.

Components are ordered by their eigenvalues, which measure how much variance each one explains.

Example: from height and weight, the first component might represent overall body size.

We usually keep only the top few components and discard the rest as noise.

In short, principal components are new smarter axes ranked by importance.

28 List three applications of PCA.

IMPORTANT

PCA is used wherever high dimensions cause trouble, and three classic applications stand out.

Application 1: visualization. Reducing 100 features to 2 lets us plot and actually see clusters in the data.

Application 2: noise reduction and faster training. Dropping low-variance components removes noise and shrinks the data models must process.

Application 3: compression. Images, sensor readings and gene data are compressed into fewer components before modeling.

It is also used for anomaly detection, since points far from the main components look suspicious.

In all cases the idea is the same: keep the signal, drop the excess dimensions.

PCA is often the first step tried when a dataset feels too wide to handle.

In short, PCA visualizes, denoises and compresses high-dimensional data.

29 Write sklearn code for PCA with 2 components.

IMPORTANT

PCA in sklearn takes just a few lines, but standardization must come first.

Code:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
X_scaled = StandardScaler().fit_transform(X)
X_pca = PCA(n_components=2).fit_transform(X_scaled)
```

StandardScaler gives every feature mean 0 and standard deviation 1.

PCA(n_components=2) keeps the two most important components.

The result X_pca has only 2 columns and can be plotted or fed into a model.

Remember: fit the scaler and PCA on training data only, then transform test data with the same objects.

In short, scale first, then reduce to 2 components.